TP – Classe Arbre Binaire
Indications

Classe Nœud – Méthode estPresente

Version concise (on a évité des if … else)

```
/**
 * Détermine si la valeur argument est présente dans l'arbre débutant au
 * noeud courant
 * @param aChercher paramètre contenant la valeur à chercher
 * @return un booléan égal à vrai ssi la valeur a été trouvée
 */
public boolean estPresente(T aChercher) {
    if (aChercher.compareTo(valeur) == 0) {

        // la valeur du noeud courant est égale à celle recherchée
        return true;
    }
    if (aChercher.compareTo(valeur) < 0) {

        // il faut chercher dans le sous-arbre gauche
        return arbreGauche != null && arbreGauche.estPresente(aChercher);
    }

    /*
     * else : on a aChercher > valeur
     * il faut chercher dans le sous-arbre droit
     */
    return arbreDroit != null && arbreDroit.estPresente(aChercher);
}
```

Classe Nœud – Début de la méthode qui affiche l'arbre

```
/**
 * Affiche les valeurs contenues dans l'arbre débutant au noeud argument
 * Chaque fois que l'on descend d'un niveau dans l'arbre, les valeurs des
 * noeuds sont affichées (en colonne) à droite des précédentes.
 * @param niveau niveau de profondeur du noeud courant. Cette valeur sert à calculer
 *               sur quelle colonne il faut effectuer l'affichage
 */
public void afficheArbreNiveau(int niveau) {

    /*
     * on affiche d'abord le sous-arbre gauche, avec un niveau de
     * profondeur égal à niveau+1
     */
    if (arbreGauche != null) {
        arbreGauche.afficheArbreNiveau(niveau + 1);
    }
```

```
        // on laisse des espaces avant d'afficher le noeud courant
         // TODO

        /*
         * on affiche ensuite le sous-arbre droit, avec un niveau de
         * profondeur égal à     ????
         */
        // TODO
    }
```

Classe ArbreBinaire

Il y a un seul attribut dans la classe :

```
    /**
     * Noeud racine de l'arbre binaire (peut être égal à null)
     */
    private Noeud<T> racine;
```

Un exemple de méthode :

```
    /**
     * Détermine si la valeur argument est présente dans l'arbre binaire
     * @param aChercher valeur à rechercher dans l'arbre
     * @return un booléen égal à vrai ssi la valeur est présente dans l'arbre
     */
    public boolean estPresente(T aChercher) {
        if (racine == null) {

            // l'arbre est vide : la valeur n'est pas présente
            return false;
        }

        // sinon : recherche à partir du noeud racine
        return racine.estPresente(aChercher);
    }
```